CETP: complete task list

Companion to ROADMAP.md, which explains *why*. This is the *what*, in order. Tier 0 is integrity work — do it before Review #2. Tiers 1–2 are what make the results good. Tier 3 is what makes it research. Tier 4 is what makes it a product.

Tier 0 — Integrity. ✅ COMPLETE (2026-10-12)

All verified numbers now live in `results/CANONICAL.md`, regenerated by `make results`. Determinism confirmed: two consecutive runs are byte-identical.

- [x] **Repointed all 9 scripts** that read the buffer-bug dataset to `data/corrected/tpch_dataset_corrected_v2.csv`. `fix1_rows_processed.py` still reads `_corrected.csv` because it *generates* `_v2`.
- [x] **Quarantined** `data/raw/` → `data/raw_buggy_do_not_use/` with a README explaining the bug and the dataset lineage. Notebook reference repointed too.
- [x] **Headline re-verified on corrected data with a leakage-free classifier:** analytical **21.61%** vs naive-linear **23.05%**, improvement **+1.44pp**, 95% cluster-bootstrap CI **[0.32, 2.56]**, excludes zero. The original [0.30, 2.55] was computed on buggy buffers; it survives.
- [x] **One canonical results script** — `scripts/analysis/canonical_evaluation.py`, run by `make results`, writes `CANONICAL.md`, `canonical_metrics.json`, `canonical_measurement_cv.csv`, `canonical_leakage_audit.csv`.
- [x] `results/phase7_report.md` **marked SUPERSEDED** with a banner explaining that its opposite conclusion was correct for the single-core signal.
- [x] **macro-F1 corrected to 0.375**, independently reproduced; `io` recall **0.000**; trivial always-predict-`mixed` baseline **0.161**. The repo (incl. the dashboard) was already right; only the external LLD says 0.562.
- [x] **Machine count = 5**, confirmed from the data. The "7" exists only in the external deck.
- [x] **Leakage audit done and quantified** — `results/canonical_leakage_audit.csv`.
- [x] **q-error added** (median and p95) alongside MAPE everywhere.
- [x] **MdAPE + per-held-out-machine + per-bottleneck-class breakdowns** added.
- [x] **Pinned** `requirements.txt` **and** `pyproject.toml` created (there were none).
- [x] **scikit-learn pinned exactly**, and artifact provenance recorded in `models/PROVENANCE.json` + `models/MODEL_CARD.md`.
- [x] **Determinism audit** — `n_jobs=1` on the RandomForest, `random_state=42` throughout, seeded bootstrap RNG. `make verify-determinism` passes.
- [x] **Repo root cleaned**; `.gitignore` extended.
- [x] **Per-cell measurement CV is now a stored column.**
- [x] **One numbers-of-record file** — `CANONICAL.md` / `canonical_metrics.json`. README points at it; `DOC_CORRECTIONS.md` lists every external-document fix.

What the audit found that was not on the original list

- [x] **A cold-cache warmup defect.** Repetition 1 of every cell is a cold run — the slowest of 20 in **61 of 105 cells**, by up to **8.33x**. `make results` now discards it. **p50 is unaffected** (worst case 0.7%), so the headline stands. But **p95 inflated >5% in 17/105 cells and p99 in 46/105, up to 6.92x** — so *every p95/p99 result published before this audit is contaminated*, and this is the most likely cause of the reported quantile under-coverage (80.7% at nominal 95%). The models were partly being asked to predict a measurement artifact.
- [x] **Classifier features are machine-invariant.** On the corrected dataset all six (`cost`, `shared_hit`, `shared_read`, `total_buffers`, `io_ratio`, `rows`) are *identical across all five machines* for a given query. Two consequences: (a) the LOMO leak was real but structurally harmless, since held-out rows duplicate training rows; (b) six of the thirteen learned-model features are **a one-hot encoding of `query_id` in disguise**, so under LOMO the GBR can memorise per-query scaling.
- [x] **Three protocols now reported, and the contrast is a result in itself.** The analytical formula is protocol-invariant (**21.61 / 21.66 / 21.64%** for LOMO / LOQO / LOMQO) while the learned model swings **17.29% → 26.86%** purely on whether it has seen the target hardware. Given every machine it is the most accurate approach by a wide margin; given a new machine it is worse than the baseline. This is the interpolation-not-extrapolation finding, now measured directly rather than inferred from one fold.
- [x] **The analytical gain is localised.** It comes almost entirely from `compute`-labelled queries (22.63% → 17.55%) and `mixed` (23.04% → 21.58%). On `bandwidth` and `io` queries it is identical to the baseline *by construction*, because the io weight is applied to `1/bandwidth_ratio`.
- [x] **RESOLVED 2026-10-13: the shipped models were stale three ways** — trained on the buffer-bug dataset (disagreeing with a correct classifier on **20 of 105 cells**, probability deltas up to 0.997), pickled with scikit-learn 1.6.1 against a 1.8.0 pin, and `hardware_signature.pkl` / `gate/cetp.yml` carrying the **single-core** compute signal. All three are fixed: retrained on `data/corrected/tpch_dataset_corrected_v2.csv` under sklearn 1.8.0, and both the signature and `gate/cetp.yml` now carry the multi-core aggregate, so **the deployed gate implements the headline finding**. The compute score is obtained by running `scripts/collection/hw_signature.py` (surfaced in the extension as `cetp.measureMachine`), which was the open product decision. Provenance in `models/PROVENANCE.json`; previous artifacts in `models_backup/`. `canonical_evaluation.py` now *verifies* this consistency at runtime rather than asserting it, so the claim cannot go stale again.
- [x] **RESOLVED 2026-10-13: experiments are separated from releases.** `phase5_classifier.py` and `phase6_scaling_regressor.py` now write to `models_candidate/`, and promotion is the deliberate `make promote-models` step, which backs up to `models_backup/` and **refuses to promote if the locked tests break**. Running the analysis can no longer silently replace what the gate ships.
- [x] **Two pre-existing breakages fixed**, both of which meant published results could not be regenerated in a pinned environment: `pre_pipeline_validation.py`'s CHECK 3 shelled out to `PROJECT_DIR/phase6_scaling_regressor.py`, but that file lives in `scripts/analysis/` — the subprocess always failed, stdout was empty, and the parse raised `StopIteration`, so **the reproducibility check had never actually run despite being reported as PASS**. And `confidence_abstention.py` called `np.trapz`, removed in NumPy 2.0, so the risk-coverage results could not be regenerated at all. All four pre-pipeline checks now genuinely pass.

Tier 1 — Make the method defensible. ✅ COMPLETE (2026-10-12)

Full results: **TIER1_RESULTS.md**. Headline: the method's central structure does not hold up, and Tier 1 found exactly why — the two-axis hardware signature is one axis measured twice (r = +0.958, PCA 97.9%/2.1%), which forces the bottleneck taxonomy to be inert.

- [x] **17. Elasticity rig built** — `scripts/collection/elasticity_probe.py`, 3 axes (parallel DOP, work_mem, bandwidth antagonist), 338 measurements, `results/elasticity_raw.csv`. Also wrote `membw_antagonist.c`. The 8-thread antagonist level was collected and **discarded**: on an 8-core machine it starves PostgreSQL of CPU rather than contending for bandwidth (q1 ran 526,827 ms against a ~4,000 ms baseline). No affinity control on macOS, so `e_bw` is a two-point slope and indicative only.
- [x] **18. Elasticity replaces the hand labels — hypothesis REFUTED.** The measured elasticities contradict the labels (compute- and bandwidth-labelled queries have *identical* measured compute sensitivity; `mixed` has the highest bandwidth sensitivity), but as weights they score 21.76% — behind the hand labels (21.62%) and a flat uniform split (21.55%). Better description, no predictive gain.
- [x] **19. Classifier ablation — the classifier is decorative.** Uniform 0.25 weights (21.55%) beat the learned classifier (21.61%). `cmp_only` (20.06%) beats everything. Oracle bound 17.07%.
- [x] **20. Conformalized quantile regression, group-calibrated** — delivers 97.0% empirical at nominal 95% where the shipped GBR gives 84.6%. Costs ~29% wider intervals.
- [x] **21. Weighted conformal under covariate shift** — implemented; **no measurable gain**. With 4 calibration machines there are only 4 distinct weights.
- [x] **22. Derive the confidence threshold — CANNOT BE DONE at n=5.** The score takes one value per machine: 6 operating points total, and risk does not fall monotonically as coverage drops. Separately: the gate helps the learned model (1.71x error ratio), is **inverted** for the analytical formula (0.86x) and flat for `cmp_only` (1.00x) — it guards the one component the project does not recommend shipping.
- [x] **23. Three-state gate — RETIRED.** A conformal gate's PASS state delivers its guarantee (1.37% breach at nominal ≤5%) where the shipped gate's does not (6.24%), but **BLOCK is empty at every SLA tested**: these interval widths can guarantee "safe" and never "unsafe". Ship a two-state guaranteed-pass / defer gate.
- [x] **24. Log-space modelling** — better point estimates, *worse* intervals (72.9% at nominal 95%) because a global residual quantile does not adapt. Use log space for points, conformal for bounds.
- [x] **25. Plan-structure features — CLEAR POSITIVE RESULT.** 50 features from `extract_plan_features_v2.py`. On unseen queries (LOQO, the honest test): 17.29% → **13.29%**, a 23% relative gain. And **the shipped 13-feature set is worse than two hardware ratios alone** (17.29% vs 16.10%). Invest here; delete the classifier.
- [x] **26. "Are two scalars enough?" — you do not have two.** r = +0.958 between the axes, 97.9% of variance on one principal component, and that axis is largely core count (r = +0.841). Unanswerable on this data. `hw_probe_suite.c` (7 axes) is written and validated (its vector-FP figure matches `compute_probe.c` to 1.8%; latency hierarchy monotonic) but has run on one machine only.
- [x] **27. arm64 laptop as dev — IT FAILS.** 324% MAPE laptop→cloud against 20% within the EC2 hull, and `naive` (prod = laptop) beats it at 123%. Root cause found: the compute score is probed at

`nproc` but a query can use only `1 + max_parallel_workers_per_gather = 3`; DOP-matching halves the error. **This impugns the headline result**, which rests on a score probed at `nproc`.

- [x] **28. Q15 fixed.** CTE with `NOT MATERIALIZED` reproduces the spec view form's plan *exactly* (same node count, depth and total cost to the cent); plain `MATERIALIZED` gives a different, ~2x cheaper plan and would have been the wrong substitution. `results/q15_resolution.md`. The workload is 22 queries.
- [x] **29. Measurement noise propagated** — floor is **0.60%** relative SE against ~20% model error. **Noise is not the binding constraint.**

Also found in Tier 1

- [x] **The 21 query files were lost** (they lived outside version control). Reconstructed in `scripts/collection/queries/` and validated against the fingerprints the originals left in `results/plan_structure_features.csv`: **plan shape exact 21/21, output cardinality exact 19/21**. q11 uses `KENYA` not `GERMANY`; q18 uses `> 314` not `> 300`. q16 and q20 remain **approximate** and are flagged in every derived artifact.
- [ ] **The STREAM-triad bandwidth probe is lost and not recoverable.** Half the hardware signature — and the basis of the `naive-linear` baseline every result is compared against — has no reproducible instrument, so no new machine can be placed on that axis. This blocked evaluating the project's own baseline laptop→cloud. **Resolution requires relaunching the five EC2 instances and re-probing the whole signature with `hw_probe_suite.c`.**
- [x] **RESOLVED by `operator_probe.py`: probe at the achievable DOP, not `nproc`** — a definition bug that is absorbed at 2–4 cores and catastrophic at 8.

Tier 2 — Dataset scale. ✅ COMPLETE except the two AWS/disk items (2026-10-12)

Full results: **TIER2_RESULTS.md**. Datasheet: **DATASHEET.md**.

The workload is now 234 queries, all validating: TPC-H 22/22, TPC-DS 99/99, JOB 113/113.

- [x] **30. One-command collection harness** — `scripts/collection/collect.py`. Parameterised over workload, concurrency, Postgres port/version and machine; records every planner-relevant setting per row. Used for every collection below. The AWS driver half is written but untested, since AWS was ruled out.
- [x] **31. IMDSv2 token bug fixed** — plus a hard assertion that the reported instance type matches the requested one (the old code silently inferred identity from `nproc`, which cannot distinguish instance types that share a vCPU count), and probing at **both** the achievable DOP and `nproc` per Tier 1's item-27 finding.
- [ ] **32. 30+ machines — BLOCKED BY AWS.** The account's EC2 vCPU quota is **1.0** in both ap-south-1 and us-east-1, below AWS's default of 5.0, and no increase was ever requested. No instance this needs can launch. Spot pricing is $0.031–0.058/hr, so 30 machines for ~3h is about **$5** — money was never the constraint. The AMI `cetp-tpch-base-v1`, the `cloud` key pair and its `.pem` all survive, so collection can run the moment a quota increase lands. **Requesting it is the single highest-value action available.**

- [x] **33. 234 queries, all validating.** TPC-DS generated locally with `dsdgen` (needed `-std=gnu89` — the kit is pre-C99 K&R C); a Postgres dialect was added since none ships. 92/99 planned unmodified, the 7 failures being the two documented incompatibilities (unaliased derived tables; a SELECT alias referenced inside an ORDER BY expression), both fixed semantics-preservingly with an **idempotent** rewriter. JOB needed no fixes at all and is the project's only **non-synthetic** workload — real IMDB skew and correlation, which TPC-H's uniform data lacks.
- [ ] **34. SF10/SF100 — traded away** for query diversity on 31 GB of disk (the author's call, and the right one: Tier 1's best result is limited by query count, not scale-factor coverage).
- [x] **35. Concurrency: 1, 2, 4, 8 clients.** Median elasticity **+0.752**, range **+0.286 to +0.984** — 8 clients buy 2.07x throughput. **The range is the finding**: some queries scale nearly freely, others serialise completely, so no single global contention factor works. Sanity check passed: 0/22 plans changed with concurrency, as they should not.
- [x] **36. Postgres matrix: 14.19 / 15.17 / 17.11 / 18.6**, byte-identical data and collation, only the engine differing. **12 of 22 queries get a different plan on at least one version**, and PG15 — the version every project measurement used — is the **slowest**, with 17/18 about **30% faster**. That is the same order as the hardware effect being predicted, so "production" is a pair (hardware **and** version) that a two-scalar hardware signature cannot express.
- [x] **37. Raw plan JSON stored**, with a structural signature per repetition and storage of any plan whose shape changes — so plan drift is detectable in the data itself. This is what the concurrency sanity check and the version comparison are built on.
- [x] **38. Packaged, not published.** `package_dataset.py` → `dist/cetp-datasets.tar.gz` plus `MANIFEST.json` with SHA-256 over 161 files, and `DATASHEET.md` in the Gebru et al. structure with every known defect listed. Third-party data (IMDB, JOB queries, tpcds-kit) is excluded by design, with source URLs and checksums recorded instead. **Publishing needs the author's Zenodo/HuggingFace account** — and `data/external/tpcds-kit/EULA.txt` must be checked before the generated TPC-DS SQL is redistributed.

Incidental obstacles worth remembering

- `pg_dump` must be >= the server version, and a **PG15 dump cannot be restored into PG14 at all** (downgrades unsupported) — which is why every version is loaded from source CSVs.
- **PostgreSQL 17+ on macOS refuses to start with an unset locale** ("postmaster became multithreaded during startup"). `LC_ALL=C` fixes it, and setting it uniformly keeps collation — and therefore sort costs — identical across versions.
- macOS ships **bash 3.2**: no associative arrays.

Tier 3 — Research. ✅ COMPLETE (2026-10-12)

Full results: **TIER3_RESULTS.md**. Related work first, per item 44 — and it changed the framing before any experiment ran: **RELATED_WORK.md**.

Headline: three of the four research angles do not survive contact with the data, and knowing which is the contribution. Every one fails for the same reason — five machines and a one-dimensional hardware signature — which makes that a finding supported from five independent directions rather than a limitation.

- [x] **39. N1 plan drift — IS the headline (corrected 2026-10-13).** A causal decomposition with forced plans (`results/tier4_plan_drift_causal.md`) finds the pure plan effect is **1.673x** against **1.145x** for settings — **70.5%** of the runtime difference — over 30 verified cells. That supersedes the observational ~16% below, which was diluted by averaging over pairs where the plan never changed. Caveat: forcing verified on only 41% of differing-plan cells, and the failures are not random. Superseded detail from the first pass: Measured on one machine varying only the planner's hardware *beliefs*, so capability is constant and drift is isolated with no confound. **20/22 queries change plan** (median 3, max 6), driven most by core count (55% of queries when `max_parallel_workers_per_gather` goes 2→8). But it is the **smaller term**: same-plan pairs swing runtime 2.16x, different-plan pairs 2.49x, so only **~16%** of variation is plan change (p = 1.7e-3). And the correlation with error is **inverted** (r = –0.551, p = 0.010): plan-volatile queries are predicted *better*, because volatility marks parallelisability. Survives as: drift is pervasive, core count drives it, and **plan volatility is a usable dev-side signal for which queries transfer**.
- [x] **40. N2 elasticity as a descriptor — refuted in Tier 1.** Better description of the queries, no predictive gain (21.76% vs 21.55% for a flat uniform split).
- [x] **41. N3 budgeted cold start — done, and it inverts the premise. One well-chosen query (13.20%) beats twelve random ones (13.45%).** Calibrating from a single *random* query makes things **worse** than not calibrating (19.51% vs 15.84%), which the old arbitrary-12 loop could never have detected. `elasticity` and `dopt` degrade as k grows because both chase extremes; for estimating one scalar you want representative queries.
- [x] **42. N4 conformal risk control — machinery works, guarantee voids. 16 of 18 configurations exceed their stated risk budget on the held-out machine**, worst fold 26.8% against a 5% budget. Not a bug: CRC assumes exchangeability, and a held-out machine is not exchangeable. Tier 1 found the same from the coverage side. Keep the framing (a *derived* threshold beats the hand-set 0.6), report the budget as aspirational.
- [ ] **43. N5 learned hardware embeddings — BLOCKED**, needs 30+ machines. Tier 1 already answered the underlying question more cheaply: with r = +0.958 between the two axes and 97.9% of variance on one component, there is no second dimension to embed.
- [x] **44. Related work written before the experiments** — and it changed the framing, exactly as the item predicted.
- [x] **45. Citations verified — four of eight were wrong or incomplete.** "QPPNet" is not the paper's own name; the covariate-shift conformal paper is Tibshirani, **Barber, Candès & Ramdas**; Ganapathi's title and Neo's eight-author list were off. Roofline's volume/issue still needs confirming.
- [x] **46. Differentiator stated — and narrowed.** The list missed the closest prior work: **COSTREAM (Heinrich, Binnig, Kornmayer & Luthra, ICDE 2024)**, a learned cost model taking hardware as input. It *does* evaluate extrapolation beyond its training range, so **"no prior work tests cross-hardware extrapolation" must not be claimed.** Only three things survive as novel: relational OLAP rather than streaming (where a plan enumerator exists at all), **latency percentiles** as the decision variable rather than a point cost with q-error percentiles, and **abstention**. Note Binnig co-authors both nearest papers.

What Tier 3 changes about the project

- [ ] **Write a measurement study, not a better predictor.** Target **aiDM or DBTest at SIGMOD**. A VLDB full paper claiming state-of-the-art is not supportable by a 5-machine evaluation and should

not be attempted.

- [ ] **Lead with the critique that generalises**: encoding hardware capability as a scalar probed at `nproc` is a definition bug that every paper in this space is exposed to and none discusses; and two correlated probes are not two dimensions. COSTREAM uses four hardware features and never tests their independence either.

Tier 4 — Product. Mostly done (2026-10-12)

An installable package now exists: `pip install -e .` gives a `cetp` command, and `cetp selftest` exercises redaction, prediction, regression detection and calibration end to end. Usage in **QUICKSTART.md**.

- [x] **Reframe to regression detection** — `cetp regress` and `cetp/regress.py`. Compares one query before/after a change so the hardware term appears in both numerator and denominator and largely cancels; `hardware_term` reports exactly how much did not, and `trust` grades the answer accordingly. This is the mode the docs recommend.
- [x] **Position as performance CI** — `.github/workflows/cetp-regression-check.yml` runs on pull requests touching `.sql`, redacts plans before upload, writes a job summary, and exits non-zero on a regression.
- [x] **Package it** — `pyproject.toml` with `cetp` and `cetp-redact` entry points. Runtime dependencies are **empty**: the shipped predictor uses the analytical formula plus `models/calibration.json`, not a pickle, so it is immune to scikit-learn version coupling. Research extras are under `[research]`.
- [x] **Plan redaction** — `cetp/redact.py`, two levels, with `assert_clean()` that re-scans a redacted plan and raises if any literal survives. Verified against a real plan and a PII torture case (emails, SSNs, numeric ids all stripped; structure preserved).
- [x] **Hardware signature registry** — `models/hardware_signatures.json` + `scripts/collection/hw_signature.py`. Seven axes, probed at the achievable DOP, with host facts, PostgreSQL settings, a source checksum and a validation verdict. `cetp.signature.ratios()` **refuses** to mix registry and legacy scales.
- [x] **VS Code extension measures the machine** — new `cetp.measureMachine` command runs the probe and saves a profile, replacing a "compute score" text field no human could fill. TypeScript compiles clean.
- [x] **Separate experiments from releases** — `phase5_classifier.py` and `phase6_scaling_regressor.py` now write to `models_candidate/`; `make promote-models` backs up, promotes, and **fails if the locked-value tests break**.
- [x] **Market the abstention** — `cetp predict` prints its own expected error and every applicable warning; README and QUICKSTART both lead with what it cannot do.
- [ ] **Production ingestion** from `auto_explain` / `pg_stat_statements` — the self-calibration loop is still seeded from a CSV.
- [ ] **Per-repo calibration state and multi-tenancy** in the measurement store.
- [ ] **Dashboard** — still the static research snapshot; make it live or delete it.
- [ ] **Postgres 13–17 plan-parser compatibility** — the matrix exists (14/15/17/18 all load) but the parser has not been exercised across it.

- [ ] **Production eval telemetry** — log predicted vs actual for every gated query.
- [ ] **Later**: MySQL, DuckDB, cloud warehouse sizing.

Tier 5 — Writing and comms. Partly done (2026-10-12)

- [x] **Document the confidence gate as implemented** — IMPLEMENTATION_REFERENCE.md §2, including that the 0.6 threshold was never derived, has only 6 operating points, and is *inverted* for the formula the project recommends shipping.
- [x] **Document the analytical weights as implemented** — §3. Notably the `io` term is applied to `1/bandwidth_ratio`, **not** to 1.0 as the LLD claims, which is why there is no io-class improvement by construction.
- [x] **Invented claims catalogued for deletion** — §4: `m_95` / `m_99` margins never existed; the artifact filenames are wrong; the table is `measurements`, not `captured_runs`. A `cetp.*` package now exists but with a different shape than the LLD invented.
- [x] **Reproducibility artifact** — `Dockerfile` + `make artifact` / `make artifact-verify`, which asserts the headline has not drifted.
- [x] **Pre-register the negative result** — PREREGISTRATION.md, with every secondary hypothesis's rejection outcome committed in advance.
- [x] **Venue decided** — VLDB *Experiments & Analysis* track, with aiDM/DBTest as fallbacks. See CONFERENCE_PLAN.md.
- [ ] **Rewrite the deck and LLD** from the verified numbers — the corrections they need are listed in DOC_CORRECTIONS.md and IMPLEMENTATION_REFERENCE.md, but the documents themselves are outside this repository.
- [ ] **Fix the acknowledged fabrications** — Appendix C dates, Appendix D section numbers, the guide designation. Yours to fix.
- [ ] **arXiv**, then the venue.

Tier 6 — Make the actual thesis work: per-operator cross-environment prediction (2026-10-13)

Context: the project's stated goal is "predict production runtime from the developer's machine", and the one experiment that tested exactly that scored **324% MAPE (153% after the DOP fix) against 123% for assuming production equals development**. The model was worse than doing nothing. Design and status: **OPERATOR_MODEL.md**.

- [x] **Move 1 — operator capability probe, in SQL.** `scripts/collection/operator_probe.py`. Six axes (seqscan, hashjoin, sort, hashagg, nestloop, parallel_seqscan). Replaces FP-vector GFLOPS, which measured a quantity TPC-H never issues. Being SQL it runs on RDS/Aurora/Cloud SQL, which the C probe could not. The axes decouple: work_mem 64MB->1MB left seqscan at 1.049x and cut hashjoin to 0.580x, a 1.81x spread no single scalar can express.
- [x] **Two guards, both of which caught real faults.** Operator verification caught the nestloop probe silently measuring a hash semi-join (5.2M reported against 1.25M true). The repeatability gate (`MAX_CV = 0.15`) caught probing under load: running the probe while the container sweep ran

moved seqscan's cv from 0.008 to 0.337 and the parallel speedup from 2.47x to 1.14x. Unreliable records are refused by `cetp.cross_env.load_rates`.

- [x] **Move 2 — per-operator scaling.** `cetp/plan_nodes.py` + `cetp/operator_model.py`. Weighted per-axis capability ratio. Three corrections that are easy to get wrong: `Actual Total Time` is cumulative (subtract children) AND per-loop (multiply by `Actual Loops`); the model predicts a *ratio* rather than summing scaled node times, because under a Gather the self-times sum to CPU time (measured 2.85x wall on q19); and the parallelism correction keys on **under_gather**, not `Parallel Aware`.
- [x] **The parallelism axis was missing and it mattered.** Found by running the model, not by reading it. Predicting the 1-CPU container from the 8-core host, TPC-H q19, true ratio 3.011x: serial axes only **1.080x (64.1% error)**; parallelism keyed on `Parallel Aware` **1.493x (50.4%)**; keyed on `under_gather` **2.664x (11.5%)**. This is the `nproc`-vs-achievable-DOP bug in a new form.
- [x] **Move 3 — the plan comes from production.** `scripts/collection/predict_prod.py`. `EXPLAIN` without ANALYZE on prod executes nothing; that plan is forced on dev with pg_hint_plan and measured there, so plan divergence (70.5% of the cross-environment difference when the plan changes) is removed by construction. Forcing failures are reported as `plan_forced=False` with trust LOW, never hidden.
- [x] **Move 4 — few-shot calibration.** `fit_calibration` (median, not mean) and `select_calibration_queries` (spread, not random). The cold-start study: 15.84% uncalibrated, 13.20% at k=1 well-chosen, but **19.51% at k=1 random** -- worse than not calibrating.
- [x] **Orthogonal capability grid RUN.** `container_grid.py` reduced to 4 CPU x 2 memory (512m dropped: TPC-H SF1 does not fit under shared_buffers=128MB + work_mem, so that cell measured OOM behaviour). Operator probe wired in per cell, plus a `--probe-only` pass so the probe can be re-run without redoing collection.
- [x] **36 unit tests** (`tests/test_operator_model.py`), each pinning a bug found by running the code. `cetp selftest` now exercises the operator path too.

Open

- [x] **Accuracy claim RESOLVED, and it splits.** `make operator-results` ran on all 8 cells, 1232 pairs (`results/operator_model.md`). **The predictor now works: 28.18% MAPE against 72.52% for assuming production equals development**, +44.33pp CI [+35.18, +54.19]. And a signature WITHOUT the parallelism axis is *worse than doing nothing* (75.86% single-scalar, 80.27% per-operator, against naive 72.52%) -- which reproduces the laptop->EC2 failure under controlled conditions. **But per-operator weighting is NOT justified**: against a single scalar given the same parallelism information it gains only +3.51pp, CI [-0.99, +8.93], spanning zero. Same verdict as the roofline against `1/compute_ratio`, reported the same way. The axis-dispersion table says why: a CPU-quota grid varies the serial axes only 1.22-1.45x while parallel speedup varies 4.45x, so it cannot discriminate move 2.
- [x] **Signature dimensionality FIXED.** Operator rates: PC1 **56.7%**, mean |r| 0.457, effective rank **4**. The old two-scalar signature: PC1 97.9%, r +0.958, rank 1. This was the blocker behind every Tier 1 negative result.
- [x] **Probe measurement duration was a real bug, found by running it.** At 400k rows the parallel scan finished in 18 ms, so worker startup dominated: cv 0.50 and an impossible **1.81x speedup on a**

1-CPU quota. At 2M rows the same cell reports **0.808x** -- parallelism correctly makes it *slower* when 3 workers share one core.

- [x] **Cardinality term was wrong and is fixed.** The first implementation used `r*log2(2r)`, giving **12.0x at r=4** against the true n-log-n ratio of ~4.5x -- a 2.7x over-penalty on every sort. Now uses the exact `r*(1+log r/log n)`.
- [x] **CORRECTED: the first laptop->prod verdict was a probe-hygiene artifact.** An earlier pass, run on probes taken while the machine was busy, reported zero-shot at 47.36% vs naive 41.11% -- i.e. *worse than doing nothing* -- and credited few-shot calibration with rescuing it (19.80% at k=2). Re-probing all 8 cells on an idle machine moved 3 cells from failing the repeatability gate to passing it (3/8 -> 6/8 reliable) and **reversed the verdict**. Probe hygiene is load-bearing.
- [x] **THE THESIS IS ACHIEVED.** Laptop (M1, arm64) -> 6 container production machines, 132 pairs (`results/laptop_to_prod.md`): **zero-shot 28.54% MAPE vs naive 38.22%.** Concentrated where capability differs: 1-CPU cells 62.58% -> 25.18% and 59.78% -> 28.84%. Measuring 5-8 queries on production gives ~23%.
- [x] **Move 4 (few-shot) is MODEST, not essential.** 28.54% -> 23.25-23.37% at k=5-8. And **k=1 is worse than not calibrating (33.74%)** -- one observation is a noisy estimate of a machine's correction, exactly as the cold-start study found. Calibrate with >=5 queries or not at all.
- [x] **Move 2 SPLIT by ablation, and this is the key scientific result.** `scalar_localized_par` -- ONE serial capability scalar, with the parallelism correction applied only to the plan's parallel wall-time share -- scores **22.23%** against the full per-operator model's **22.30%**: delta **-0.08pp, CI [-0.73, +0.44], NOT significant.** They are indistinguishable. Meanwhile applying parallelism whole-query instead of localising it costs **+6.91pp, CI [+0.95, +14.73], SIGNIFICANT. So the contribution is plan-structure decomposition, not per-operator capability.** Per-operator RATE weighting adds nothing measurable, and the dispersion table says why: serial axes span only 1.02-1.13x across the grid while parallel speedup spans 3.66x. This is a SIMPLIFICATION result -- one serial scalar + one parallel speedup + the plan's parallel/serial split is sufficient on this evidence.
- [x] **The zero-shot residual is DIAGNOSED.** Systematic over-prediction (mean signed error +0.181, per machine +0.015 to +0.359). The probe is cache-resident (~120 MB tables vs a 128 MB buffer pool) while TPC-H reads ~850 MB past it. Over-prediction correlates with each query's I/O fraction (Spearman r = +0.302).
- [ ] **ADD AN I/O AXIS TO THE PROBE.** Highest-value change remaining: nothing in the signature measures read throughput past the buffer pool. Needs a probe table several times `shared_buffers`.
- [x] **Second ISA OBTAINED, and the result is NEGATIVE** -- the most important finding of this round (`results/cross_isa_operator_rates.md`). An x86_64 AMD EPYC 9V74 runner was probed through `.github/workflows/x86-operator-probe.yml` (reps=9, record passed the repeatability gate). Crossing **arm64 -> x86_64 perturbs the operator mix by 1.365x**; the worst pair **within** the arm64 container grid perturbs it by **1.371x**. The ISA boundary is no harder on operator mix than a CPU quota change on one machine. **This refutes the stated motivation for per-operator capability modelling**, and means the 59.7% cross-ISA floor is NOT caused by operator-mix divergence. Combined with the grid ablation (-0.08pp, CI [-0.73, +0.44]), two independent experiments now agree: per-operator RATES are not the useful part; the parallelism term and its plan-structure localisation are.
- [x] **CROSS-ISA PREDICTION MEASURED, and it fails without calibration** (`results/cross_isa_prediction.md`). 176 pairs over 8 production machines, one a real x86_64

AMD EPYC 9V74 collected through GitHub Actions. **Same ISA (7 arm64 cells, 154 pairs): operator 29.31% vs naive 37.39% -- the model works. Cross ISA (1 x86 cell, 22 pairs): operator 72.84% vs naive 21.41% -- 3.4x WORSE than doing nothing.** Mechanism: the probe says x86 is 1.667x slower than the M1; it is actually 1.044x. Not operator-mix divergence (refuted above) but the ABSOLUTE SCALE -- tuples/s on a synthetic operator is not comparable across microarchitectures. The same defect as the original FP-GFLOPS signature; five architecture-sensitive scalars instead of one did not fix it. **Calibration repairs it and is mandatory across an ISA:** k=5 takes the x86 cell to 23.38% (floor 16.75%) -- but only to PARITY with naive's 21.41%, not past it.

- [ ] **Make calibration mandatory in the product when the two machines report different `client_platform` architectures.** Uncalibrated cross-ISA is the one configuration measured to be far worse than doing nothing, now three times (EC2 324%/160%, x86 72.84%). `cetp predict-prod` should refuse, or at minimum hard-warn, on an uncalibrated cross-architecture pair.
- [ ] **A second x86 machine** is the cheapest next measurement -- one machine establishes the direction and size of the failure, not a distribution.
- [ ] **Find what DOES explain the residual.** Ranked candidates: storage/buffer-pool regime (EC2 was EBS, the laptop is local NVMe; over-prediction already correlates with per-query I/O fraction at r=+0.302, and no probe axis measures storage); per-query variance of 18-43x WITHIN a single EC2 machine, which no per-machine quantity can express; engine build and config differences.
- [ ] **Vary settings, not just quotas**, to create plan divergence and make move 3 testable.
- [ ] **Validate the cardinality term** with a second scale factor.
- [x] **Concurrency is now an explicit caller-supplied input.** `predict_scaling(..., prod_clients=N, concurrency_elasticity=e)` applies `N^e`; `DEFAULT_CONCURRENCY_ELASTICITY = 0.734` is the measured median and a default halves the error at 8 clients (73.8% -> 41.8%), while the per-query oracle reaches 6.7%. `prod_clients=1` emits a loud SINGLE-CLIENT warning, because a silent single-client assumption is how this term went ignored for the whole project. Fitted and cross-validated in `results/concurrency.md`; the honest negative is that elasticity is NOT predictable from a plan (LOQO MAE 0.191 vs 0.167 for the median).